

Spark-to-Paper: End-to-End Research Paper Generation as a Composable Skill

Zhuoyang Qian^{1,*}, Biao Wu^{2,*}, Yiran Wang^{1,*}, Chris D Yan^{1,*},
Desan Dai¹, Liangwei Zheng¹, Jin Jiang¹, Jusheng Zhang¹, Wenhao Wang^{1,‡}

¹Vast Intelligence Lab ²University of Technology Sydney

*Equal Contribution ‡Corresponding Author

<https://github.com/Spark-To-Paper-Skills/spark-to-paper-skills>
wangwenhao@vastilab.com

Abstract

Turning a research idea into a complete paper requires more than text generation: the system must retrieve literature, design and execute experiments, revise claims according to evidence, produce publication-ready figures, and maintain consistency across a long generation process. We present **Spark-to-Paper**, an end-to-end research paper generation system implemented as thirteen composable skills inside an existing coding assistant, without requiring a separate agent platform or orchestration service. Spark-to-Paper separates model-based judgment from deterministic operations that can be directly executed and checked. It further separates experiment planning from reporting, so that required evidence is specified before results are observed and manuscript claims are revised according to measured outcomes. To improve reliability over long research trajectories, the system combines deterministic integrity checks with self-critique and bounds a failure mode we call the Self-Refutation Loop, in which repeated experiments continue to reject the original research objective. Spark-to-Paper also produces editable vector figures through programmatic plotting for experimental results and code-based reconstruction for generated method diagrams. Across eight controlled research topics, Spark-to-Paper achieves 99.5% citation validity and 96.4% figure editability. A controlled ablation increases fabrication detection from 14% for a single-pass draft to 92% with the full integrity and review stack, while adversarial review achieves 74% precision. The full system uses 11.9M tokens, costs \$8.1 per manuscript, and requires 3.2 hours on average. These results show that end-to-end research paper generation can be implemented as a lightweight, composable workflow inside existing coding assistants while keeping experimental evidence central to how claims are accepted, revised, or abandoned.

1 Introduction

Turning a research idea into a complete academic paper requires substantially more than generating text. A researcher must identify relevant literature, design and run experiments, decide whether the resulting evidence supports the original hypothesis, revise claims when it does not, produce publication-ready figures, and maintain consistency across a manuscript that evolves over many stages. Recent autonomous research agents have begun to automate this broader process, demonstrating that language models can participate in ideation, experimentation, review, and paper generation [13, 28, 20, 7, 27]. However, these systems are typically implemented as standalone applications with their own orchestration layers and supporting infrastructure. This makes them powerful, but also separate from the coding environments in which much of the actual research work already takes place.

At the same time, modern coding assistants already provide many of the basic capabilities required for research automation: they can inspect project files, execute code, search for information, call external tools, and revise artifacts over long interactions [3, 2]. Existing skill-based research tools exploit some of these capabilities for tasks such as literature search, outlining, and manuscript drafting [26], but generally stop short of the complete research-to-paper process. This raises a simple question: *can end-to-end research paper generation be implemented as a collection of reusable skills inside an existing coding assistant, rather than as a separate autonomous research platform?*

We present **Spark-to-Paper**, a system that realizes this design through thirteen composable skills. Each skill handles a research task, such as planning, literature retrieval, writing, review, figure generation, or experiment execution, while all skills communicate through artifacts in a shared project directory. The coding assistant determines how each skill should be executed from the current project state, while a lightweight pipeline specifies only the high-level order of tasks. Within each skill, we further separate judgment from execution: the language model handles decisions that require interpretation or reasoning, whereas deterministic programs handle operations that can be explicitly executed and checked.

Three components are particularly important for extending this design beyond ordinary paper drafting. **First**, Spark-to-Paper separates experiment planning from experiment reporting. The evidence required by the manuscript is specified before results are observed, experiments are executed to resolve missing evidence, and the resulting artifacts are used to revise claims throughout the paper. **Second**, the system combines deterministic integrity checks with long-horizon self-critique [21, 14]. We additionally identify a failure mode that we call a **Self-Refutation Loop**, in which the system repeatedly concludes that its own experiments fail to support the original research objective and continues revising the same direction. Spark-to-Paper bounds this process, records unsuccessful trajectories as failure reports, and restarts from a new idea rather than forcing every research direction to succeed. **Third**, the figure pipeline separates quantitative plots from explanatory figures. Experimental plots are generated directly from measured data, while method figures are first designed with an image-generation model and then reconstructed through code into editable vector PDFs.

We evaluate Spark-to-Paper through controlled experiments, retrospective audits, and qualitative case studies. Across eight controlled research topics, the full system achieves 99.5% citation validity and 96.4% figure editability. In a controlled ablation with injected unsupported claims, fabrication detection increases from 14% for a single-pass draft to 92% with the complete integrity and review stack, while adversarial review achieves 74% precision on raised issues. The full system uses 11.9M tokens on average, costs \$8.1 per manuscript, and requires 3.2 hours of wall-clock time in our instrumented runs. We report these measurements together with audits and disclosed costs from prior systems, while avoiding direct price equivalence where the underlying workloads differ.

Our contributions are threefold:

- We introduce Spark-to-Paper, an end-to-end research paper generation system implemented as thirteen composable skills inside an existing coding assistant, without requiring a separate agent platform or orchestration service.
- We develop mechanisms for evidence-grounded long-horizon research generation, including pre-committed experiment design, evidence-guided claim revision, deterministic integrity checks, bounded recovery from self-refutation loops, and code-reconstructed editable figures.
- We provide a controlled evaluation of the resulting research artifacts, including citation validity, fabrication detection, figure editability, review precision, robustness, and generation cost, together with ablations and qualitative analyses of system behavior and failure modes.

2 Related Work

2.1 Autonomous End-to-End Research Agents

A growing line of work runs an entire research cycle, *i.e.*, ideation, experimentation, writing, and often self-review, as a standalone autonomous system. Lu et al. [13] and its successor Yamada et al. [28] generate, run, and write up machine learning experiments with a simulated review stage; Liu et al. [12] extends this to a self-reinforcing, human-in-the-loop pipeline reported at a scale of tens of thousands of lines of code; Mitchener et al. [15] and Ghareeb et al. [7] target open-ended scientific discovery over long, multi-hour autonomous runs with independently-audited statement accuracy;

Table 1: Qualitative capability comparison against directly related systems, based on each system’s own public documentation (paper or repository) as of this writing. ✓ full, ● partial, – not offered or not documented.

System	End-to-end	Runs exper.	Draws figures	Editable vectors	No standing infra.
AI Scientist / v2 [13, 28]	✓	✓	✓	–	–
AutoResearchClaw [12]	✓	✓	✓	–	–
Kosmos / Robin [15, 7]	●	✓	●	–	–
Idea2Story [27]	–	–	●	–	✓
ARS [26]	●	–	–	–	✓
CycleResearcher [25]	●	✓	–	–	–
Spark-to-Paper (ours)	✓	✓	✓	✓	✓

Gottweis et al. [8] and Schmidgall et al. [20] similarly pair an LLM-driven experimentation loop with a writing stage. These systems match or exceed Spark-to-Paper’s breadth, particularly in autonomously proposing and running novel experiments, but each ships as a standalone application, typically a dedicated codebase with its own orchestration server, and in several cases a graph database or cluster scheduler, rather than as skills that run inside an existing coding assistant with no additional service to operate.

2.2 Lightweight, Skill-Based Writing Assistants

A second line of work stays inside a coding assistant’s existing skill or plugin mechanism rather than shipping a separate application. Xu et al. [27] and Wu [26] decompose literature search, drafting, and review into composable steps much as Spark-to-Paper does; Elovic [5] and Karpathy [10] automate a narrower slice, *i.e.*, web-grounded report writing, and single-GPU experiment iteration, respectively; Chen [4] surveys this emerging category directly. Weng et al. [25] pairs an automated researcher with an automated reviewer trained specifically to critique it, the closest existing analogue to Spark-to-Paper’s own adversarial review stage. None of these tools, however, runs real experiments end to end or produces editable vector figures rather than embedded bitmaps, the two capabilities at the center of this paper’s contributions.

2.3 LLM Agents, Tool Use, and Self-Critique

Spark-to-Paper’s internal mechanisms draw on, without directly reimplementing, a broader agent literature. Its reasoning-then-acting stage structure follows the interleaved reasoning-and-action pattern of Yao et al. [30]; its use of external tools (web search, a DOI resolver, a plotting library) for tasks a language model should not do itself echoes Schick et al. [19] and Nakano et al. [17]; its refine and review stages apply a model’s own critique of its output in the manner of Shinn et al. [21] and Madaan et al. [14]; its planning stage decomposes a paper into an outline before any prose is written, in the same spirit as deliberate multi-step reasoning approaches [24, 29]; and its persistent, file-backed working state across skill invocations resembles the memory-and-reflection architecture of Park et al. [18], adapted here to a single long-running document rather than a simulated society of agents.

2.4 Grounded Generation and Citation Integrity

The concern motivating Spark-to-Paper’s gate infrastructure is well documented: large language models fabricate citations and factual claims at measurable, non-trivial rates when asked to write about

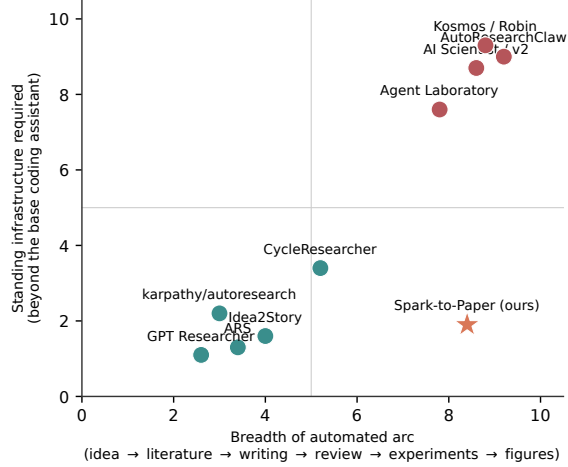


Figure 1: Qualitative positioning against directly related systems, on two axes derived from Table 1. Coordinates are the authors’ own ordinal assessment from each system’s public documentation, not a measured benchmark.

literature they have not actually retrieved [23, 9, 31]. Retrieval-augmented generation is the standard mitigation [11, 22, 6], and Spark-to-Paper’s citation stage is best understood as a retrieval-then-verify instance of this pattern, specialized to bibliographic metadata rather than passage-level facts. Its no-fabrication rule for tabular results connects to a related but distinct literature on generating text from structured data faithfully [16], where the risk is not an invented source but an invented number. Taken together, this body of work establishes both ends of the space Spark-to-Paper occupies: systems that automate more of the research process but require standing infrastructure to do it, and systems that stay infrastructure-free but automate a narrower slice of it. Positioning a new system between two established poles is easy to assert and harder to substantiate, since it invites exactly the kind of overclaim this paper otherwise argues against; Table 1 and Fig. 1 are our attempt to make that positioning checkable against each compared system’s own public documentation rather than leaving it as a rhetorical claim. The following sections substantiate this positioning in mechanistic detail and, in Sections 5 and 7, examine the trade-offs that accompany this design.

3 Spark-to-Paper

Spark-to-Paper is a collection of thirteen skills that runs inside an existing coding assistant. Each skill handles a specific research task, such as planning a paper, searching for literature, writing and revising the manuscript, reviewing claims, generating figures, or running experiments. Rather than building a separate agent system, Spark-to-Paper uses the coding assistant’s existing abilities to read files, execute code, search for information, and call external tools. The skills share a common project directory. Each skill reads the artifacts produced so far, performs its task, and writes its output back to the project. For example, the planning skill produces a paper blueprint, the citation skill builds a verified bibliography, and the writing skill uses both to generate the manuscript. Later skills operate on the same manuscript, figures, and experimental outputs. The project files therefore provide a simple interface between skills and allow the paper to evolve across the full pipeline. In our implementation, Spark-to-Paper runs inside Claude Code using a model from the Claude family [1, 3, 2]. The design itself is not tied to Claude Code. A coding assistant with comparable abilities to inspect project files, use tools, and execute code can support the same skill-based design.

Skills as the execution unit. A skill defines what a research task should accomplish, the constraints it must satisfy, the tools it may use, and the artifacts it should produce. It does not specify every reasoning step needed to complete the task. Instead, the coding assistant decides how to carry it out from the current project state. For example, the refinement skill may only reorganize an argument in one paper, while another may require additional citations or changes after a failed consistency check. Both use the same skill. This differs from encoding the full system behavior as a fine-grained agent graph, where individual actions and transitions must be specified in advance. Spark-to-Paper nevertheless maintains a simple high-level order for paper generation. The pipeline specifies *which task comes next*, while each skill determines *how that task is completed*. This keeps the process easy to follow without fixing the detailed execution path of every research task.

Model and deterministic tools. Within each skill, we separate tasks that require judgment from tasks that can be executed and checked mechanically. The language model handles decisions such as how to organize an argument, which literature is relevant, whether available evidence supports a claim, and how a result should be presented. These decisions depend on the manuscript and the evidence available at that point in the project. Operations with explicit rules are handled by deterministic scripts. Examples include checking manuscript structure, validating citations, compiling L^AT_EX, plotting measured results, and checking generated files. These operations do not benefit from open-ended generation and can instead be executed reproducibly and verified directly. The resulting design follows a simple division of responsibility: the model handles judgment, while code handles operations that can be explicitly executed and checked.

3.1 From Input to Paper

As shown in Fig. 2, the thirteen skills provide the reusable building blocks of Spark-to-Paper. Their execution is organized by the `ts-paper` orchestrator into an input-routing step, seven core paper-generation stages, and an experiment stage. Additional skills support particular inputs or stages, such as expanding a short research idea, processing measured results, or converting generated figures into editable formats. Thus, the number of skills does not correspond one-to-one with pipeline stages.

Stage 0: Input Routing. Spark-to-Paper first inspects the user input and determines where the pipeline should begin. A short research idea can first be expanded into a structured research proposal, whereas an already developed proposal can enter the paper pipeline directly. The system also determines whether real experimental results are available. This decision selects one of two result-integrity modes. In **Proposal Mode**, the input contains no measured results. The system may design experiments and prepare result tables, but unavailable values must remain unspecified. In **Data-Aware Mode**, measured data or experimental outputs are available, and quantitative statements in the manuscript must be supported by those results. This mode is propagated through the remaining stages so that the same generation pipeline can operate both before and after experiments have been conducted.

Stage 1: Planning. The planning skill converts the input into a structured paper blueprint. It identifies the research question, main contributions, section structure, notation, and experimental design. It also reads the specification of the target venue to determine structural requirements such as section organization and length constraints. The resulting blueprint becomes the main specification used by later stages.

Stage 2: Citation. The citation skill builds the bibliography required by the planned paper. It searches for relevant literature and verifies candidate references using bibliographic information such as DOI records, arXiv identifiers, and other metadata. The goal is to find verifiable support for the claims and context required by the blueprint rather than to generate references from model memory. Verified references are stored in a BibTeX file and reused throughout the manuscript.

Stage 3: Writing. The writing skill generates the complete L^AT_EX manuscript from the blueprint and verified bibliography. All sections are written against the same project context so that terminology, notation, contributions, and experimental claims remain consistent across the paper. The selected integrity mode also controls quantitative writing: Proposal Mode leaves unavailable results unspecified, while Data-Aware Mode allows measured values supported by the provided data.

Stage 4: Refinement. The refinement skill revises the manuscript as a whole rather than treating sections independently. It removes repetition, reconciles terminology and notation, improves local arguments, and adjusts section lengths to the target template. Deterministic checks are then rerun to ensure that revision has not introduced structural, citation, or integrity errors.

Stage 5: Review. The review skill challenges the current manuscript before it is finalized. Multiple isolated review passes examine complementary aspects of the paper, such as technical soundness, experimental design, and the strength of the evidence. Review comments are tied to specific passages in the manuscript and are checked before being accepted as valid issues. Confirmed problems are returned to the refinement stage for correction. We describe the corresponding quality-control mechanisms in Section 5.

Stage 6: Figure Generation. The figure skill generates figures according to their role in the paper. Plots that represent measured results are produced programmatically from the underlying data. Method diagrams, system architectures, and other explanatory figures may instead be produced with an image-generation model. Generated raster figures are subsequently processed into editable artifacts when possible, while preserving their approved visual content. The figure pipeline is described in detail in Section 6.

Stage 7: Assembly. The assembly skill combines the manuscript sections, bibliography, figures, and venue template into a complete L^AT_EX project. Template-specific operations such as section ordering, citation formatting, and document structure are derived from the template specification rather than hard-coded for a single venue. The project is then compiled, and deterministic checks are used to verify that the final manuscript contains no unresolved citations or L^AT_EX errors.

Stage 8: Experiment Execution. Once a complete first draft is available, Spark-to-Paper checks whether the planned experiments can be executed using the available code and data. When they are feasible, the experiment skill runs them, records the measured outputs, and uses the results to update the manuscript. Result tables and data-dependent figures are regenerated from the measurements, and claims in the abstract, introduction, experiments, and conclusion can be revised to match the new evidence. If the required code or data are unavailable, the system leaves the corresponding results unspecified rather than generating values. The experiment stage therefore connects the proposal and data-aware parts of the pipeline: the earlier stages specify what evidence the paper needs, while

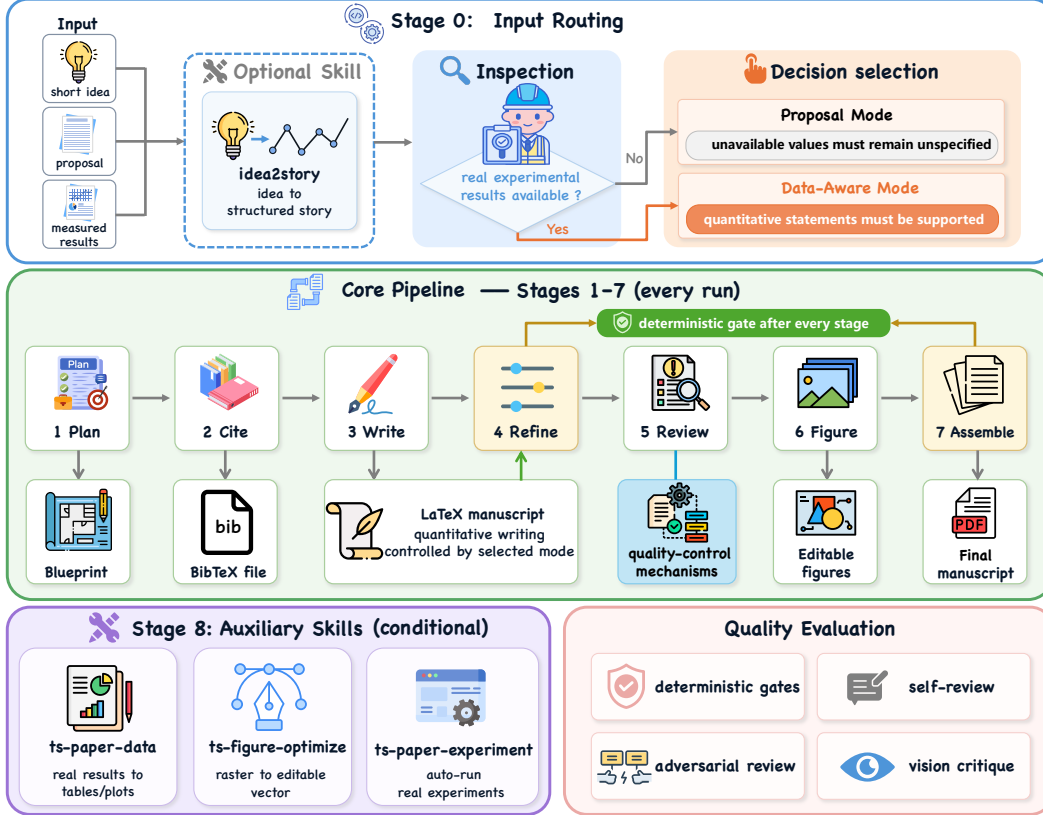


Figure 2: Overview of Spark-to-Paper execution. Stage 0 selects the result-integrity mode, Stages 1–7 coordinate through persistent project artifacts, and conditional Stage 8 writes measured evidence back into the manuscript. Model judgment governs context-dependent decisions, while deterministic tools execute checkable operations.

this stage obtains that evidence when the required resources are available. We describe experiment execution in detail in Appendix B.

3.2 Key Components

The pipeline above summarizes the overall execution of Spark-to-Paper. We leave most implementation details, including individual skill specifications, intermediate artifacts, and auxiliary checks, to Appendices D and A. In the main text, we focus on three components that are particularly important for end-to-end paper generation: generating publication-ready and editable figures, planning and executing experiments based on the evidence required by the manuscript, and maintaining reliable reasoning over long generation horizons through repeated self-critique and claim revision. We describe these components in the following sections.

4 Experiment Design and Revision

Spark-to-Paper separates experiment planning from experiment reporting to reduce the risk of adapting the evaluation protocol after observing the results. Before execution, the planning stage specifies the datasets, baselines, metrics, ablations, and result tables required to evaluate the manuscript’s main claims. The table structure is fixed in advance, while numerical cells remain empty until the corresponding experiments are completed. This creates a lightweight form of preregistration within the pipeline: the system decides what evidence would be required before seeing that evidence, and the experiment stage inherits this design rather than redefining it based on observed outcomes.

The experiment stage then maps manuscript claims to the evidence needed to support them and identifies which experiments are still missing. Rather than exhaustively running every experiment

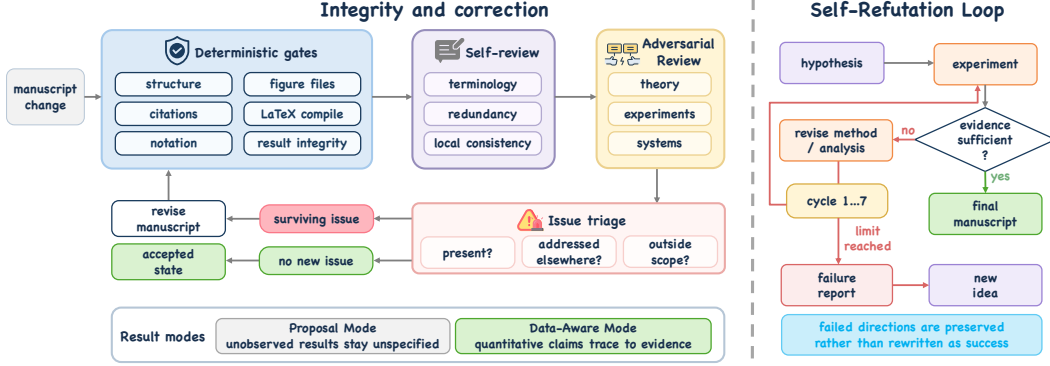


Figure 3: Integrity and correction in Spark-to-Paper. Deterministic gates enforce verifiable properties, Self-Review and Adversarial Review challenge semantic decisions, and surviving issues trigger revision. Experiment–critique–revision cycles are capped at seven; an unresolved trajectory becomes a failure report rather than a successful manuscript.

suggested by a generic template, Spark-to-Paper executes the minimal set needed to resolve these evidence gaps. Paper-type-specific guidance may recommend suitable baselines, metrics, ablations, and common failure checks, but does not override the committed design. Expensive runs or external data acquisition must be justified by the specific claim they support and by the availability of the required resources. Experimental execution produces traceable artifacts, including logs, metric files, tables, and figures, rather than manuscript claims directly. A numerical result is allowed to enter the paper only when it can be traced back to its dataset, model configuration, seed, metric, and source output.

After execution, Spark-to-Paper revisits the manuscript according to the resulting evidence. Claims are classified as SUPPORTED, PARTIALLY-SUPPORTED, UNSUPPORTED, CONTRADICTED, or NEEDS-CONFIRMATION, and are retained, weakened, removed, moved to limitations, or used to trigger additional experiments accordingly. Null, negative, or inconclusive results are preserved rather than omitted, and the resulting changes are propagated across the manuscript so that the abstract, introduction, results, and conclusion remain consistent with the final evidence. Design commitment and several artifact-level checks are machine-checked, while claim-level evidence diagnosis is currently performed by the model and recorded in structured reports. We provide the full claim admission protocol and additional implementation details in Appendices C and D.

5 Integrity and Self-Critique

Long-horizon paper generation introduces two different types of errors. Some are explicit and machine-checkable, such as broken citations, missing figures, compilation failures, or numerical results appearing without supporting data. Others are semantic and emerge as the manuscript evolves: an early claim may become inconsistent with later evidence, an argument may drift across sections, or a decision made earlier in the pipeline may no longer remain appropriate. Spark-to-Paper handles these two cases differently. Deterministic checks enforce constraints that can be stated explicitly, while model-based self-critique revisits decisions that require semantic judgment. Together, they provide hard checks for verifiable properties and iterative correction for errors that emerge over long generation horizons.

5.1 Deterministic Integrity Checks

For properties with explicit correctness criteria, Spark-to-Paper uses deterministic gates. The current implementation checks the paper blueprint, citations, manuscript content, figures, and the compiled L^AT_EX project, covering requirements such as template structure, citation consistency, notation, required figure files, and successful compilation. The same mechanism also enforces result integrity. In **Proposal Mode**, experimental results that have not been observed must remain unspecified rather than being filled with generated values; in **Data-Aware Mode**, quantitative statements must be supported by provided data or experimental outputs. A stage proceeds only after its applicable checks pass. Because these conditions can be stated and verified directly, they are implemented as

deterministic programs rather than model judgments. Detailed gate definitions and failure conditions are provided in Appendix A.

5.2 Self-Critique over Long-Horizon Generation

Deterministic checks cannot determine whether an argument is sufficiently supported, whether different sections remain conceptually consistent, or whether an earlier claim should be revised after new evidence becomes available. Spark-to-Paper therefore uses two levels of model-based critique. **Self-Review** operates locally after an edit, checking the modified content against the surrounding manuscript and repairing terminology drift, redundancy, and local inconsistencies. **Adversarial-Review** operates at the manuscript level, where multiple isolated review passes examine the paper from complementary perspectives, including theoretical soundness, experimental design, and systems validity. Every proposed issue must identify and quote the specific passage it challenges, and is then checked from three directions: whether the problem is actually present, whether it has already been addressed elsewhere in the manuscript, and whether it falls outside the stated scope of the work. Issues that can be refuted are discarded, while those that survive are returned for revision. Review continues until an additional round produces no new surviving issues.

Together, these steps form a long-horizon correction loop: the system makes a claim or writing decision, later challenges it, attempts to refute the challenge, and revises the manuscript when the challenge remains valid. This allows later evidence to change earlier decisions rather than simply being appended to them. For example, experimental results may weaken an earlier claim and trigger revisions to the experiments section, introduction, abstract, or conclusion. A similar principle is used for figures, where reference grounding and visual review compare generated diagrams against the manuscript and revise them when necessary (Section 6).

5.3 Self-Refutation Loops and Bounded Recovery

A failure mode we observe in long-horizon research generation is a **Self-Refutation Loop**. The system begins with a research hypothesis or design objective, plans experiments to test it, and then evaluates whether the observed results provide sufficient evidence for the intended claim. When the evidence is judged insufficient, the model may revise the method, experimental design, or analysis and run another round of experiments. In some cases, however, the new evidence is again judged insufficient, causing the system to repeat the same cycle of experimentation, critique, and revision. This behavior is different from ordinary iterative improvement. A productive iteration resolves a concrete weakness or produces new evidence that moves the research toward a clearer conclusion. In a self-refutation loop, the system repeatedly concludes that its own results do not adequately support the original hypothesis or design objective, yet continues revising the same research direction rather than accepting that the idea may simply not work. The manuscript and experiments may therefore continue to change without converging to a defensible scientific conclusion.

Spark-to-Paper does not assume that such loops can always be resolved. Instead, we bound the number of experiment–critique–revision cycles to seven. If the research objective still cannot be supported after this limit, the current trajectory is terminated. The system records the original idea, the attempted methods and experiments, the observed results, and the reasons why the evidence was insufficient in a failure report. This report is treated as the outcome of an unsuccessful research direction rather than being rewritten into an apparently successful paper. The system then starts a new research trajectory from a different idea and reruns the full pipeline. Some trajectories may fail in the same way, while others may produce experimental evidence that supports a coherent set of claims. Only the latter are consolidated into a final manuscript. In this way, Spark-to-Paper does not force every research idea to succeed: it bounds self-refutation loops, preserves failed attempts as research outcomes, and moves on to alternative ideas when the current hypothesis cannot be supported.

6 Editable Figure Generation

Spark-to-Paper generates paper figures through two different paths depending on their role. Method and explanatory figures are first created visually with an image-generation model and then reconstructed as editable vector graphics through code. Figures that report experimental results are instead generated directly from measured data using plotting programs. This separation allows the system

Role-aware figure generation

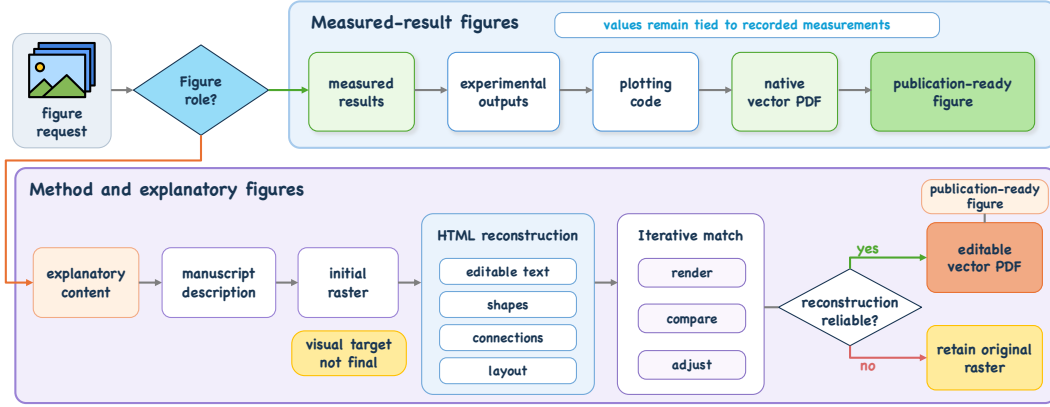


Figure 4: Role-aware editable figure generation. Measured results follow deterministic plotting to native vector PDF, whereas explanatory content uses a raster visual target followed by iterative HTML reconstruction and vector export; unreliable reconstruction falls back to the raster.

to use generative models for visual design while keeping quantitative figures strictly grounded in experimental results.

6.1 Method and Explanatory Figures

Method figures, system diagrams, and other explanatory illustrations are generated directly from the manuscript. The image-generation model reads the relevant method description and produces an initial raster figure that captures the intended structure, layout, and visual presentation. We use this generated image as a visual target rather than as the final paper artifact. Spark-to-Paper then uses the coding capabilities of the underlying assistant to reconstruct the figure in HTML, using editable text, shapes, connections, and layout elements. Because the first reconstruction may differ from the generated reference, the system renders the HTML and compares it with the original image. It then iteratively adjusts the layout, geometry, text placement, and visual elements to reduce the discrepancy. In practice, a small number of correction rounds is usually sufficient to recover the main structure and appearance of the generated figure.

Once the reconstruction is accepted, the HTML representation is rendered to PDF. Text and graphical elements created during reconstruction remain editable and vector-based, producing a publication-ready figure without manually redrawing the original image. The generated raster therefore serves primarily as a visual specification, while the reconstructed HTML/PDF provides the final artifact used in the manuscript.

6.2 Experimental Result Figures

Figures that report experimental results follow a different path. Because their content corresponds directly to measured values, Spark-to-Paper does not use an image-generation model to create them. Instead, the system reads the experimental outputs and writes plotting code for the required visualization, such as performance comparisons, ablation studies, or trend plots. The plotting program generates the figure directly from the recorded measurements and exports it as a PDF. This keeps the visualized values tied to the underlying experimental results while also producing a vector-format artifact that can be included directly in the manuscript. Thus, generative image models are used for figures whose primary purpose is visual explanation, whereas quantitative figures are generated deterministically from data.

7 Evaluation Protocol

We evaluate Spark-to-Paper using three complementary sources of evidence: controlled experiments, retrospective analysis of existing outputs, and qualitative case studies. Together, these evaluations examine whether Spark-to-Paper can complete the end-to-end research pipeline and produce

Table 2: Evaluation dimensions and corresponding measurements used to assess the quality and generation efficiency of Spark-to-Paper.

Metric	Measurement
<i>Quality</i>	
Citation validity	Resolved references / total references
Fabrication detection	Detected unsupported claims / injected claims
Figure editability	Editable elements / total figure elements
Review precision	Verified issues / raised review issues
Cross-template robustness	Successful templates / supported templates
<i>Efficiency</i>	
Generation cost	Monetary cost, tokens, wall-clock time, and deployment footprint

Table 3: Main comparison of Spark-to-Paper with human-written preprints, prior autonomous research systems, and a single-pass LLM baseline across artifact quality and generation efficiency metrics.

System	Citation exist. (%)	Fig. editability (%)	Tokens (M)	USD	Wall-clock
Human-written preprints (sampled)	97.8 [94.6, 99.4]	58 [44, 71]	n/a	n/a	n/a
AI Scientist, released papers [13]	93 (42/45)	0 (0/210)	n/r	\$10–15 (amort.)	~12 h / batch
AI Scientist-v2, workshop set [28]	91 (58/64)	3 (0–8)	n/r	~\$20–25 / attempt	≤15 h / run
Agent Laboratory, released paper [20]	96 (27/28)	0 (0/30)	n/r	\$2.33 (gpt-4o)	~19 min
Single-pass LLM draft (same backbone)	81 (range 76–86)	n/a	0.11 (0.09–0.13)	\$0.66 (0.55–0.76)	16 min (13–19)
Spark-to-Paper (full stack)	99.5 [98.4, 100]	96.4 [92.7, 98.6]	11.9 [10.2, 13.7]	\$8.1 [6.9, 9.6]	3.2 h [2.6, 3.9]

manuscripts that are reliable, editable, and efficient to generate. Our evaluation focuses on two questions: **whether Spark-to-Paper produces higher-quality research artifacts than comparable systems, and whether it does so at lower cost.** Whenever possible, comparisons with prior systems are based on their publicly released papers and artifacts, avoiding the need to rerun their pipelines. Our own runs are instrumented directly, while cost figures for prior systems are taken only from their reported results.

To avoid evaluating a component with the same mechanism used to optimize it, we use independent evaluation procedures whenever possible. For example, citation validity is verified after generation using external bibliographic metadata services rather than the citation checks used inside the pipeline. We also report the provenance of each result and whether optional in-loop checks were enabled during generation.

7.1 Evaluation Dimensions

We evaluate Spark-to-Paper along six dimensions, summarized in Table 2. The first five measure artifact quality, while the last measures generation efficiency. For the controlled evaluation, research topics are selected independently of system outputs, and the topic set, model configuration, and evaluation criteria are fixed before generation begins. We register the evaluation protocol with an external timestamp and report all outcomes regardless of whether they favor Spark-to-Paper. The full system is evaluated on eight externally selected topics, three of which are also shared with the single-pass baseline for paired comparison. We report uncertainty across topics rather than relying on individual runs. For prior systems, we use only values explicitly reported in their papers or repositories and mark unavailable measurements as *not reported* rather than estimating them.

7.2 Main Results

Table 3 compares Spark-to-Paper with human-written preprints, previously released autonomous research systems, and a single-pass LLM baseline. Spark-to-Paper achieves the highest citation validity among the evaluated systems, with 99.5% of references successfully resolved, while preserving 96.4% of figure elements in editable form. In comparison, the released papers from prior autonomous research systems exhibit citation validity between 91% and 96%, but provide little or no figure editability. The single-pass baseline is substantially cheaper and faster, but its citation validity drops to 81%, illustrating the quality-efficiency trade-off introduced by the full pipeline.

Table 4: Ablation study of the Spark-to-Paper quality stack, showing the contribution and incremental cost of gating, self-review, and adversarial review.

Configuration	Fabr. detection (%)	Review precision (%)	Δ tokens (M)	Δ USD
Single-pass draft (no gates)	14 (5/36) [6, 29]	n/a	ref.	ref.
Gates only	69 (25/36) [53, 82]	n/a	$+8.1 \pm 0.9$	$+5.3 \pm 0.5$
Gates + self-review	81 (29/36) [65, 90]	n/a	$+1.1 \pm 0.2$	$+0.6 \pm 0.1$
Gates + self-review + adversarial review	92 (33/36) [78, 97]	74 (42/57) [61, 83]	$+2.6 \pm 0.4$	$+1.6 \pm 0.2$

Per-paper output, seven end-to-end runs reported in the project’s own showcase documentation

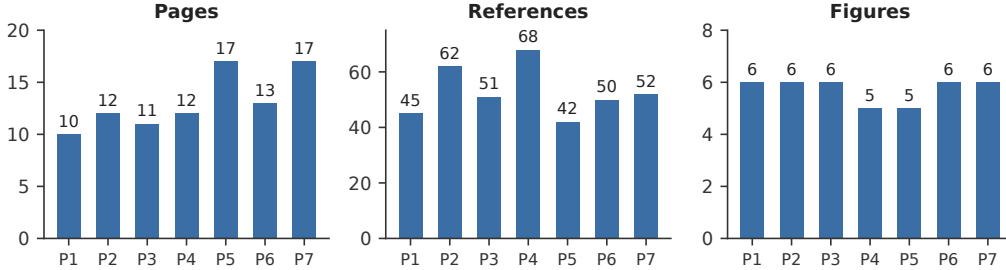


Figure 5: Page, reference, and figure counts for the end-to-end papers documented in the project’s own showcase materials. Self-reported by the system’s maintainers, not an independent measurement; included as existence evidence, not as a benchmark result.

The comparison combines controlled measurements with retrospective audits. Results for Spark-to-Paper and the single-pass baseline are obtained from our instrumented runs on pre-registered topics. The full system is evaluated on eight topics, while three of these topics are additionally run with the single-pass baseline for paired comparison. Human-written preprints serve as a reference point for artifact quality and are evaluated using the same external citation and figure-analysis procedures. By contrast, results for AI Scientist, AI Scientist-v2, and Agent Laboratory are obtained by auditing artifacts publicly released by the corresponding systems; we do not rerun these systems or normalize them to our model backbone, topics, or pricing environment.

For our measurements, citation validity is computed over 384 references from eight Spark-to-Paper campaign papers; the human-preprint reference comprises 320 references from eight sampled papers. Figure editability for Spark-to-Paper is evaluated over approximately 1,900 ground-truth figure elements, excluding figures that are intentionally rasterized by design. Reported intervals are cluster-bootstrap intervals over papers; when only three paired runs are available, we report the observed range instead of a confidence interval. Cost and runtime for our systems are measured directly from execution logs. For prior systems, we report only values explicitly stated in their papers or repositories, using the pricing assumptions of their original publications rather than retrospectively repricing them. We use *n/a* for structurally inapplicable quantities and *n/r* for quantities not reported by the original system.

7.3 Ablation Study

Table 4 isolates the contribution of the major components in Spark-to-Paper’s quality-control stack. Starting from a single-pass draft, adding deterministic and model-based gates increases fabrication detection from 14% to 69%. Adding self-review further raises detection to 81%, while the complete stack with adversarial review reaches 92%. The adversarial-review stage also achieves 74% review precision, indicating that most issues raised by the reviewer correspond to verifiable problems rather than spurious critiques. These gains come with additional inference cost: relative to the preceding configuration, gates add the largest token and monetary overhead, while self-review and adversarial review provide further quality improvements at smaller incremental cost.

We evaluate fabrication detection using a fixed corpus of 36 seeded probes spanning ten failure families and drawn from three sources. All probes are injected into the shared source material before drafting at the same intervention point, and the identical probe set is used for all four configurations. The detection procedure is also held fixed across configurations. Consequently, differences between rows reflect changes in the quality-control stack rather than changes in either the injected failures

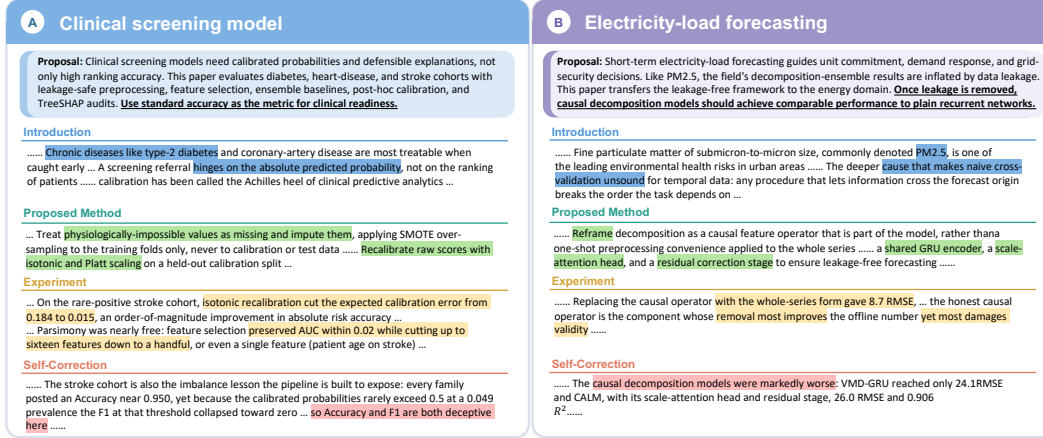


Figure 6: Case study on two different domain demo papers with only one short proposal as inputs, incorrect expectations are highlighted by Bold and Underline text.

or the evaluation instrument. Detection intervals are Wilson 95% confidence intervals over the 36 probes.

Review precision is evaluated separately because it is defined only for configurations containing an explicit review stage. We sample 60 issues raised during review and have them assessed by blinded raters. Three “cannot tell” judgments are excluded, leaving 57 issues in the reported denominator. The resulting precision therefore measures the fraction of review findings that correspond to independently verifiable issues.

The final two columns quantify the incremental computational cost of each added layer. Token and dollar deltas are measured on the three topics shared across configurations and are reported relative to the immediately preceding row. These increments are therefore intended to show the marginal cost of each quality-control component, rather than to reconstruct the eight-topic average reported in Table 3. Because the ablation uses the three-topic paired subset, its cumulative cost does not necessarily equal the full-system mean in the main comparison. We use *n/a* when a metric is undefined for a configuration and *ref.* for the single-pass reference configuration.

7.4 Case Study

We show two case studies in Fig. 6 that demonstrate Spark-to-Paper’s ability to autonomously identify research gaps, synthesize novel methods, and maintain academic integrity. The generated answers also criticize the drawbacks of current approaches and highlight the drawbacks and research gaps in a specific domain. In clinical risk screening, the framework goes beyond raw discrimination accuracy to address critical miscalibration and interpretability, introducing a leakage-safe pipeline that integrates post-hoc probability recalibration with TreeSHAP driver audits. In time-series forecasting, Spark-to-Paper redefines traditional decomposition by framing it as a causal, sliding-window feature operator to eliminate look-ahead bias. Crucially, by conducting an empirical leakage audit, the system fearlessly exposes how conventional whole-series decomposition inflates accuracy by borrowing information from the future, demonstrating its commitment to evidence-based scientific truth.

Furthermore, we inject two incorrect expectations into the proposal to demonstrate the effectiveness of our model’s refinement and revision capabilities. In the clinical screening example, the proposal mistakenly positions Accuracy as the primary evaluation metric, a common pitfall in the medical domain due to severe data imbalance. As reflected by the paper’s results, the proposed method implements multiple evaluation metrics to cross-validate the trustworthiness of the final results, ultimately concluding that both Accuracy and F1 scores are highly deceptive. In the PM2.5 forecasting domain, we inject another strong prior expectation from the user (that causal models would achieve comparable performance). The paper’s actual empirical results present a verifiable conflict with this claim, showing that decomposition models perform markedly worse. This scenario effectively tests the model’s ability to prioritize objective experimental evidence over misleading user prompts.

8 Conclusion

We present Spark-to-Paper, an end-to-end research paper generation system that consists of thirteen composable skills inside an existing coding assistant. Spark-to-Paper reuses the assistant’s native capabilities for file interaction, tool use, and code execution, while separating model-based judgment from deterministic operations that support direct verification. Beyond manuscript drafting, the system plans experiments around the evidence required by the paper, revises claims according to empirical results, combines deterministic integrity checks with long-horizon self-critique, and produces editable vector figures through programmatic plotting or code-based reconstruction. Our experiments show that this lightweight design produces complete research artifacts with high citation validity and figure editability while substantially improving the detection of unsupported claims. Importantly, Spark-to-Paper does not force every research trajectory to succeed: when repeated experimentation fails to support the original hypothesis, the system bounds the resulting Self-Refutation Loop, preserves the unsuccessful trajectory as a research report, and moves on to a new idea. This design makes automated research a traceable process in which the system revises or abandons claims according to evidence rather than forcing them toward a successful narrative.

References

- [1] Anthropic. The Claude 3 model family: Opus, sonnet, haiku. <https://assets.anthropic.com/m/61e7d27f8c8f5919/original/Claude-3-Model-Card.pdf>, 2024. URL <https://assets.anthropic.com/m/61e7d27f8c8f5919/original/Claude-3-Model-Card.pdf>.
- [2] Anthropic. Claude agent SDK overview. <https://code.claude.com/docs/en/agent-sdk/overview>, 2026. URL <https://code.claude.com/docs/en/agent-sdk/overview>.
- [3] Anthropic. Claude code documentation. <https://code.claude.com/docs/en/overview>, 2026. URL <https://code.claude.com/docs/en/overview>.
- [4] Deli Chen. From copilots to colleagues: A survey of autonomous research agents. https://victorchen96.github.io/auto_research/auto_research_survey.pdf, 2026. URL https://victorchen96.github.io/auto_research/auto_research_survey.pdf.
- [5] Assaf Elovic. GPT researcher. <https://github.com/assafelovic/gpt-researcher>, 2026. URL <https://github.com/assafelovic/gpt-researcher>.
- [6] Yunfan Gao, Yun Xiong, Xinyu Gao, Kangxiang Jia, Jinliu Pan, Yuxi Bi, Yi Dai, Jiawei Sun, and Haofen Wang. Retrieval-augmented generation for large language models: A survey. *arXiv preprint arXiv:2312.10997*, 2023. URL <https://arxiv.org/abs/2312.10997>.
- [7] Ali Essam Ghareeb, Benjamin Chang, Ludovico Mitchener, Angela Yiu, Caralyn J. Szostkiewicz, Jon M. Laurent, Muhammed T. Razzak, Andrew D. White, Michaela M. Hinks, and Samuel G. Rodrigues. Robin: A multi-agent system for automating scientific discovery. *arXiv preprint arXiv:2505.13400*, 2025. URL <https://arxiv.org/abs/2505.13400>.
- [8] Juraj Gottweis, Wei-Hung Weng, Alexander Daryin, Tao Tu, Petar Sirkovic, Artiom Myaskovsky, Grzegorz Glowaty, Felix Weissenberger, Alessio Orlandi, Dan Popovici, et al. Accelerating scientific discovery with co-scientist. *Nature*, pages 1–3, 2026.
- [9] Ziwei Ji, Nayeon Lee, Rita Frieske, Tiezheng Yu, Dan Su, Yan Xu, Etsuko Ishii, Yejin Bang, Andrea Madotto, and Pascale Fung. Survey of hallucination in natural language generation. *ACM Computing Surveys*, 55(12), 2023. URL <https://arxiv.org/abs/2202.03629>.
- [10] Andrej Karpathy. autoresearch: AI agents running research on single-GPU nanochat training automatically. <https://github.com/karpathy/autoresearch>, 2026. URL <https://github.com/karpathy/autoresearch>.
- [11] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 33, pages 9459–9474, 2020. URL <https://arxiv.org/abs/2005.11401>.
- [12] Jiaqi Liu, Shi Qiu, Mairui Li, Bingzhou Li, Haonian Ji, Siwei Han, Xinyu Ye, Peng Xia, Zihan Dong, Meng Chen, Congyu Zhang, Letian Zhang, Guiming Chen, Haoqin Tu, Xinyu Yang, Lu Feng, Xujiang Zhao, Haifeng Chen, Jiawei Zhou, Xiao Wang, Weitong Zhang, Hongtu Zhu, Yun Li, Jieru Mei, Hongliang Fei, Jiaheng Zhang, Linjie Li, Linjun Zhang, Yuyin Zhou, Sheng Wang, Caiming Xiong, James Zou, Zeyu Zheng, Cihang Xie, Mingyu Ding, and Huaxiu Yao. AutoResearchClaw: Self-reinforcing autonomous research with human-AI collaboration. *arXiv preprint arXiv:2605.20025*, 2026. URL <https://arxiv.org/abs/2605.20025>.
- [13] Chris Lu, Cong Lu, Robert Tjarko Lange, Jakob Foerster, Jeff Clune, and David Ha. The AI scientist: Towards fully automated open-ended scientific discovery. *arXiv preprint arXiv:2408.06292*, 2024. URL <https://arxiv.org/abs/2408.06292>.
- [14] Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegrefe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, et al. Self-refine: Iterative refinement with self-feedback. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023. URL <https://arxiv.org/abs/2303.17651>.

- [15] Ludovico Mitchener, Angela Yiu, Benjamin Chang, Mathieu Bourdenx, Tyler Nadolski, Arvis Sulovari, Eric C Landsness, Daniel L Barabasi, Siddharth Narayanan, Nicky Evans, et al. Kosmos: An ai scientist for autonomous discovery. *arXiv preprint arXiv:2511.02824*, 2025.
- [16] Nafise Sadat Moosavi, Andreas Rücklé, Dan Roth, and Iryna Gurevych. SciGen: A dataset for reasoning-aware text generation from scientific tables. In *Advances in Neural Information Processing Systems (NeurIPS) Datasets and Benchmarks Track*, 2021. URL <https://arxiv.org/abs/2104.08296>.
- [17] Reiichiro Nakano, Jacob Hilton, Suchir Balaji, Jeff Wu, Long Ouyang, Christina Kim, Christopher Hesse, Shantanu Jain, Vineet Kosaraju, William Saunders, et al. WebGPT: Browser-assisted question-answering with human feedback. *arXiv preprint arXiv:2112.09332*, 2021. URL <https://arxiv.org/abs/2112.09332>.
- [18] Joon Sung Park, Joseph C. O’Brien, Carrie J. Cai, Meredith Ringel Morris, Percy Liang, and Michael S. Bernstein. Generative agents: Interactive simulacra of human behavior. In *Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology (UIST)*, 2023. URL <https://arxiv.org/abs/2304.03442>.
- [19] Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can teach themselves to use tools. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023. URL <https://arxiv.org/abs/2302.04761>.
- [20] Samuel Schmidgall, Yusheng Su, Ze Wang, Ximeng Sun, Jialian Wu, Xiaodong Yu, Jiang Liu, Michael Moor, Zicheng Liu, and Emad Barsoum. Agent laboratory: Using LLM agents as research assistants. *arXiv preprint arXiv:2501.04227*, 2025. URL <https://arxiv.org/abs/2501.04227>.
- [21] Noah Shinn, Federico Cassano, Edward Berman, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023. URL <https://arxiv.org/abs/2303.11366>.
- [22] Kurt Shuster, Spencer Poff, Moya Chen, Douwe Kiela, and Jason Weston. Retrieval augmentation reduces hallucination in conversation. In *Findings of the Association for Computational Linguistics: EMNLP*, 2021. URL <https://arxiv.org/abs/2104.07567>.
- [23] William H. Walters and Esther Isabelle Wilder. Fabrication and errors in the bibliographic citations generated by ChatGPT. *Scientific Reports*, 13:14045, 2023. doi: 10.1038/s41598-023-41032-5.
- [24] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed Chi, Quoc Le, and Denny Zhou. Chain-of-thought prompting elicits reasoning in large language models. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 35, 2022. URL <https://arxiv.org/abs/2201.11903>.
- [25] Yixuan Weng, Minjun Zhu, Guangsheng Bao, Hongbo Zhang, Jindong Wang, Yue Zhang, and Linyi Yang. Cycleresearcher: Improving automated research via automated review. *arXiv preprint arXiv:2411.00816*, 2025. URL <https://arxiv.org/abs/2411.00816>. ICLR 2025.
- [26] Cheng-I Wu. academic-research-skills: A claude code skill suite for research, writing, and review. <https://github.com/Imbad0202/academic-research-skills>, 2026. URL <https://github.com/Imbad0202/academic-research-skills>.
- [27] Tengyue Xu, Zhuoyang Qian, Gaoge Liu, Li Ling, Zhentao Zhang, Biao Wu, Shuo Zhang, Ke Lu, Wei Shi, Ziqi Wang, Zheng Feng, Yan Luo, Shu Xu, Yongjin Chen, Zhibo Feng, Zhuo Chen, Bruce Yuan, Harry Wang, and Kris Chen. Idea2story: An automated pipeline for transforming research concepts into complete scientific narratives. *arXiv preprint arXiv:2601.20833*, 2026. URL <https://arxiv.org/abs/2601.20833>.

- [28] Yutaro Yamada, Robert Tjarko Lange, Cong Lu, Shengran Hu, Chris Lu, Jakob Foerster, Jeff Clune, and David Ha. The AI scientist-v2: Workshop-level automated scientific discovery via agentic tree search. *arXiv preprint arXiv:2504.08066*, 2025. URL <https://arxiv.org/abs/2504.08066>.
- [29] Shunyu Yao, Dian Yu, Jeffrey Zhao, Izhak Shafran, Thomas L. Griffiths, Yuan Cao, and Karthik Narasimhan. Tree of thoughts: Deliberate problem solving with large language models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023. URL <https://arxiv.org/abs/2305.10601>.
- [30] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. In *Proceedings of ICLR*, 2023. URL <https://arxiv.org/abs/2210.03629>.
- [31] Yue Zhang, Yafu Li, Leyang Cui, Deng Cai, Lemao Liu, Tingchen Fu, Xinting Huang, Enbo Zhao, Yu Zhang, Yulong Chen, Longyue Wang, Anh Tuan Luu, Wei Bi, Freda Shi, and Shuming Shi. Siren’s song in the AI ocean: A survey on hallucination in large language models. *arXiv preprint arXiv:2309.01219*, 2023. URL <https://arxiv.org/abs/2309.01219>.

A Deterministic Gates and Failure Conditions

Spark-to-Paper uses deterministic gates for properties with explicit correctness criteria. Each gate reads the artifacts produced by the current stage, emits a structured report, and returns a pass or fail decision before those artifacts are passed downstream. We distinguish between *fatal* violations, which stop execution, and *warnings*, which are recorded but do not block progress. Fatal conditions are reserved for violations that make an artifact invalid, inconsistent, or unverifiable.

The **Template Gate** and **Blueprint Gate** enforce structural consistency before manuscript generation. The former verifies that the venue specification contains the required document structure, section constraints, citation settings, template resources, and assembly information. The latter checks that the planned manuscript conforms to these requirements, including its sections, contributions, figures, tables, citation types, and title constraints. Violations with an unambiguous correction may be normalized automatically; inconsistencies that require semantic judgment cause the gate to fail.

The **Citation Gate** checks bibliographic integrity, including malformed or duplicate entries, unresolved citation keys, unused references, and inconsistencies between citations and their associated claims. When identifier resolution is enabled, DOI, URL, or preprint records are additionally verified. Deterministically invalid identifiers cause failure, whereas temporary network or service errors are reported as warnings. The **Manuscript Gate** checks structural requirements together with result integrity. It detects unresolved placeholders, invalid result-table structure, missing notation definitions, and related venue-specific constraints. Result-integrity checks follow the execution mode selected during input routing: unobserved empirical results are rejected in Proposal Mode, while quantitative statements that cannot be grounded in the supplied evidence are flagged in Data-Aware Mode. If the evidence required for verification is itself unavailable, the gate fails rather than relying on model judgment.

The **Figure Gate** verifies that required figure artifacts exist and that their representation and generation path are consistent with their intended role. Quantitative figures must remain grounded in measured results, while explanatory figures claimed to be editable must retain the corresponding editable structure. Raster output remains permissible when the content is inherently raster or when reliable reconstruction is unavailable. Finally, the **Compilation Gate** requires the assembled $\text{\LaTeX}$ project to compile successfully with citations and cross-references resolved. Compilation repair is bounded; unresolved failures are reported rather than triggering an indefinite repair loop.

These gates deliberately cover only properties that can be verified without semantic judgment. They can determine whether a required artifact exists, whether a reported value is grounded in available evidence, whether a citation resolves, or whether a project compiles. They do not determine whether a contribution is important, an argument is persuasive, an experimental design is scientifically appropriate, or a figure communicates its message effectively. Such questions are handled by the model-based review and claim-assessment mechanisms described elsewhere in the paper. Deterministic gating therefore does not attempt to automate every quality decision; instead, it makes directly verifiable requirements non-negotiable once they have been specified.

B Experiment Execution Details

Spark-to-Paper does not directly write experimental outputs into the manuscript. Instead, experiment execution is treated as an evidence-grounded process. The system first identifies the evidence required by the current manuscript claims, determines which corresponding experiments are feasible, and executes those experiments using the available code and data. The resulting measurements are then verified before they can be treated as experimental evidence. Algorithm 1 summarizes this procedure.

The feasibility decision preserves the experimental design committed to during the planning stage. Spark-to-Paper does not replace a specified evaluation with an easier proxy, a different data construction, or synthetic observations when doing so would change the experiment described by the manuscript. When a required resource is unavailable, the system records the missing dependency and leaves the corresponding result unspecified rather than substituting a generated value. This prevents the evaluation protocol from being adapted to what is easiest to execute after the intended evidence has already been specified.

Algorithm 1 Evidence-Grounded Experiment Execution

Require: Manuscript M , experimental design D , available code C , data X

Ensure: Revised manuscript M' , experiment artifacts A

```
1: Identify claims  $Q$  in  $M$  that require experimental evidence
2: Derive required experiments  $\mathcal{E}$  from  $D$  and  $Q$ 
3: for each experiment  $e \in \mathcal{E}$  do
4:   Assess whether  $e$  is feasible using  $C$  and  $X$ 
5:   if  $e$  is feasible then
6:     Execute  $e$  and record configurations, seeds, logs, and measurements
7:     Verify result provenance and experimental consistency
8:     if the result passes verification then
9:       Admit the measured result into  $A$ 
10:      Update the corresponding tables and figures
11:    else
12:      Reject the result and revise the associated claim
13:    end if
14:  else
15:    Record the missing resource
16:    Keep the corresponding result unspecified
17:  end if
18: end for
19: Re-evaluate manuscript claims using the admitted evidence
20: Propagate revisions across evidence-dependent sections
21: Preserve negative, null, and inconclusive results
22: Recompile and validate the revised manuscript
23: if the central research objective remains unsupported then
24:   Pass the trajectory to the bounded Self-Refutation procedure
25: end if
26: return  $M', A$ 
```

For feasible experiments, the system records the configuration, dataset, random seed, metric definition, logs, and raw measurements produced during execution. A measurement is accepted as verified experimental evidence only when it can be traced to these source artifacts and, for aggregated results, recomputed from the underlying runs. The verification step also checks that reported experiments were actually executed, that the implementation contains the components, baselines, and ablations described by the manuscript, and that the evaluation protocol does not contain detectable failures such as data leakage, invalid splits, or mismatched metrics. Measurements that fail these checks are rejected as experimental evidence rather than completed or repaired through generation.

C Claim Admission Protocol

Given the verified evidence produced by experiment execution, Spark-to-Paper re-evaluates manuscript claims rather than assuming that the initial narrative remains valid. Before claim-level assessment, the system identifies the primary contribution type of the paper, such as a framework, model, benchmark, dataset, system, or empirical study. This distinction matters because different claims require different forms of evidence. For example, failure of one model instance to outperform a baseline may weaken a model-level performance claim without necessarily invalidating a broader framework contribution. Claims are therefore assessed relative to the contribution they are intended to support rather than against a single result table.

Each major claim is assigned one of five evidence labels: *supported*, *partially-supported*, *unsupported*, *contradicted*, or *needs-confirmation*. A claim can be labeled *supported* or *partially-supported* only when it is linked to specific verified evidence. The assigned label determines the corresponding revision action, as summarized in Table 5.

Claim weakening changes the scope of a statement rather than merely reducing its rhetorical strength. Claims of statistical significance, robustness, generalization, or broad empirical superiority are retained only when the corresponding evidence has been obtained. When the evidence supports a

Table 5: Claim admission labels and corresponding revision actions.

Label	Action
<i>supported</i>	Retain with evidence-matched wording
<i>partially-supported</i>	Narrow the claim or request additional evidence
<i>unsupported</i>	Run a feasible missing experiment, weaken, or remove
<i>contradicted</i>	Remove or report as a limitation
<i>needs-confirmation</i>	Return the unresolved claim for author confirmation

Table 6: Main persistent artifacts used by Spark-to-Paper.

Artifact	Stage	Purpose
<code>blueprint.json</code>	Planning	Paper structure, claims, notation, experiments
<code>template.json</code>	Planning	Venue specification and execution mode
<code>refs.bib</code>	Citation	Verified bibliography
<code>claims_map.json</code>	Citation	Claim–citation associations
<code>sections/*.tex</code>	Writing	Section-level manuscript sources
<code>figures/</code>	Figure	Figures and generation records
<code>results.facts.json</code>	Data	Grounded quantitative evidence
<code>main.tex/pdf</code>	Assembly	Final manuscript project
<code>logs/*.io.md</code>	All stages	Stage-level input and output records

narrower conclusion, the manuscript is revised to state that narrower conclusion explicitly. Negative, null, and inconclusive results are preserved rather than omitted, and the overall research narrative is allowed to weaken when required by the measured evidence.

Revisions are propagated across all occurrences of the affected claim. A claim that appears in the abstract, introduction, results, and conclusion is updated consistently rather than corrected only where the corresponding experiment is reported. The abstract is revised last so that it reflects the final claim strength of the completed manuscript. Claims marked *needs-confirmation* cannot remain unresolved in the final deliverable; they must be supported, revised, removed, or explicitly resolved by the author.

The protocol separates machine-checkable evidence integrity from semantic claim assessment. Deterministic programs verify the integrity of the experimental evidence and the final manuscript, while determining whether a particular piece of evidence semantically supports a claim is currently performed by the model. Each assessment is recorded in a structured report containing the original claim, its manuscript location, assigned label, supporting evidence, revision action, and rationale. Claim admission therefore remains model-based where semantic judgment is required, but the resulting decisions are explicit and auditable rather than being hidden inside manuscript rewriting.

D Implementation Details

We provide additional implementation details on input routing, persistent project artifacts, and venue-specific configuration below.

The pipeline first routes the user input according to its current research state. A short idea may be expanded into a structured proposal, whereas a developed proposal can enter the paper-generation pipeline directly. The system also determines whether measured experimental evidence is already available and selects either Proposal Mode or Data-Aware Mode. This mode is stored in the project state and propagated to downstream stages. In Proposal Mode, unavailable empirical results remain unspecified; in Data-Aware Mode, quantitative statements must be grounded in the supplied data or experimental outputs. The same downstream pipeline can therefore operate both before and after experiments have been conducted.

Each stage materializes its output as an explicit project artifact. Planning produces the paper blueprint and active template specification; citation retrieval produces a verified bibliography and claim–citation mapping; writing produces section-level $\text{\LaTeX}$ sources; figure generation writes visual artifacts and their associated records; and assembly produces the complete $\text{\LaTeX}$ project and compiled manuscript. When measured results are available, their extracted quantitative facts are stored separately so that downstream integrity checks can distinguish observed evidence from generated text. Table 6 summarizes the main persistent artifacts used for communication between stages.

Venue-specific behavior is parameterized through the template specification rather than hard-coded into individual skills. The specification defines the section structure, length constraints, citation style, figure and table requirements, title and keyword constraints, and assembly conventions consumed by downstream stages. Supporting a new venue therefore requires a new specification and the corresponding template resources rather than changes to the pipeline logic. Template provenance is retained explicitly: official or user-provided styles can be used for submission, while bundled approximations are marked as demonstration-only and must be replaced before submission.

E Generated Paper Examples

We additionally provide several examples of papers generated by Spark-to-Paper. The following figures show representative sections from complete end-to-end outputs, including the introduction, method, experimental results, and analysis. These examples are included to illustrate the structure, level of detail, and visual form of the generated manuscripts, rather than as an additional quantitative evaluation.

Interpretable, Calibrated Early Screening of Chronic Disease Risk with Feature Selection and Ensembles

[AUTHORS TBD]^{1*}
¹ [Affiliation TBD]
[CORRESPONDING EMAIL TBD]

Abstract

Early-screening models for chronic disease are increasingly accurate at ranking who is at risk, yet two properties that decide whether a clinician can act on them are routinely neglected: the predicted risk probability is rarely trustworthy in absolute terms, and the model exposes no defensible account of why a patient was flagged. We reframe screening from a discrimination-only exercise into a calibrated, interpretable risk-estimation problem and instantiate it as one leakage-safe pipeline that imputes physiologically-impossible values, over-samples the minority class in the training folds only, selects features by random-forest permutation importance, compares gradient-boosted trees, a random forest, a support-vector classifier, and a perceptron on all features versus the selected subset, and recalibrates the designated boosted model with isotonic and Platt scaling on a held-out split, auditing the Brier score and the expected calibration error on the untouched test split. Run on the Pima diabetes, Cleveland heart-disease, and imbalanced Kaggle stroke cohorts, the kernel classifier discriminated best on the two smaller cohorts, at an AUC of 0.826 and 0.873, while the tree ensembles led on stroke; selection preserved AUC within 0.02 while cutting up to sixteen features to one; and recalibration cut the stroke expected calibration error from 0.184 to 0.015. The stroke cohort is the imbalance lesson: an Accuracy near 0.950 hides an F1 near zero at a 0.049 prevalence, so the precision-recall curve and calibration, not Accuracy, are the metrics that survive. TreeSHAP on the uncalibrated boosted score recovered clinically plausible drivers that cross-checked the selection.

1 Introduction

Chronic diseases such as type-2 diabetes and coronary-artery disease are most treatable when they are caught early, and machine-learning models built on routinely collected tabular clinical variables are an attractive front line for population screening: the measurements are already taken in ordinary care, the models are cheap to run, and they can flag who should be referred for confirmatory testing. Public benchmark cohorts have made this concrete, with routine clinical features and a recorded outcome supporting tabular diabetes-risk screening on the Pima Indians cohort [Smith et al., 1988] and heart-disease-risk screening on the Cleveland cohort [Detrano et al., 1989]. On these and similar cohorts, learned classifiers predict diabetes onset [Naz and Ahuja, 2020] and coronary-artery disease [Mohan et al., 2019] from a handful of routine variables, which is what makes tabular machine learning a credible, low-cost complement to existing screening practice.

The applied literature has pushed these models toward higher discrimination, reporting strong area under the ROC curve for diabetes [Naz and Ahuja, 2020] and heart-disease [Mohan et al., 2019] risk, but three properties that decide whether a clinician can act on the output are routinely left unaddressed. First, the models are opaque: a single risk score or an aggregate AUC tells a physician nothing about which factors drove a patient's flag, so the output is hard to trust, audit, or turn into a conversation with the patient, and interpretability of such black-box models is exactly what a coherent

Preprint.

Figure 7: Paper Demo (Part 1): Screenshot for the demo paper introduction.

2017). These works establish that calibration matters and that imbalance handling can undermine it, but the reliability of the probability is rarely measured side by side with the discrimination and the explanation it is meant to make trustworthy, which is precisely the joint measurement we report.

The single nearest line of work is the interpretable, imbalance-aware clinical pipeline that pairs over-sampling with attribution on one cohort [Tasin et al., 2023]: it selects, resamples, and explains, but it optimizes and reports discrimination and leaves the reliability of the probability unmeasured, so its explanation is attached to a risk whose absolute value is unaudited. We resolve all three gaps at once by treating discrimination, calibration, and explanation as one leakage-safe procedure, reporting reliability beside discrimination as a matter of evaluation hygiene [Steyerberg et al., 2010], and exercising the identical selection, comparison, calibration, and explanation across several real cohorts rather than one. The next section formalizes that pipeline.

3 Method

We present the unified interpretable, calibrated, imbalance-aware screening pipeline, a single leakage-safe procedure that prepares a clinical cohort, selects a compact feature subset, compares several classifier families, recalibrates the designated gradient-boosted model’s probabilities, and explains its drivers. The same procedure runs unchanged across the cohorts: feature selection decides what a classifier may read, the cross-family comparison tests whether committing to a small subset costs discrimination, probability calibration makes the predicted risk usable as an absolute number rather than a ranking, and TreeSHAP audits which retained factors govern that risk. Figure 1 shows the overall data flow. The guiding principle is that a deployable screening model must emit a probability a clinician can act on and a rationale a clinician can scrutinize, so the design prefers a smaller, calibrated, auditable model over a sharper opaque one.

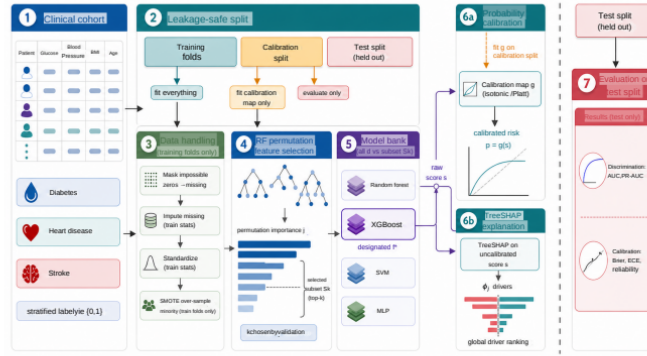


Figure 1: Overview of the unified screening pipeline: leakage-safe splitting, impossible-value handling, imputation, and training-fold-only SMOTE, random-forest permutation feature selection, the compared model families (RF, XGBoost, SVM, MLP) on all features versus the selected subset, isotonic and Platt calibration of the gradient-boosted model fitted on a held-out calibration split, and TreeSHAP explanation of the uncalibrated gradient-boosted score, with discrimination and calibration both evaluated on the untouched test split, applied identically across the clinical cohorts.

3.1 Problem Formulation

We cast early screening across the clinical cohorts as one supervised binary-classification setup with several instances. Let X be a feature matrix whose rows are patients and whose d columns are routinely collected clinical and physiological measurements, and let x_i denote the feature vector of the i -th patient. The target $y_i \in \{0, 1\}$ is a binary disease-risk label, at-risk versus not, obtained from

Figure 8: Paper Demo (Part 2): Screenshot for the demo paper method.

the mean squared error between the predicted probability $\hat{p}_i$ and the outcome y_i , for which lower is better. The Brier score, however, mixes calibration with refinement, so we report alongside it the expected calibration error

$$\text{ECE} = \sum_b \frac{n_b}{n} |acc_b - \bar{p}_b|, \quad (2)$$

the mean over equal-width probability bins b of the gap between the empirical accuracy acc_b and the mean predicted probability $\bar{p}_b$ in the bin, weighted by the bin count n_b , a calibration-specific metric that isolates the risk mis-statement Brier conflates with sharpness. Both are read on the test split. Figure 3 shows the measured effect, where recalibration moves the stroke reliability curve back toward the diagonal. The screening decision applies a one-half operating point to the calibrated probability $\hat{p}$, a natural cut once probabilities are calibrated, fixed in advance without consulting the test set; the threshold-dependent Precision, Recall, and F1 are reported at this point, and for the highly imbalanced cohort we additionally report the area under the precision-recall curve, more informative than F1 at low prevalence.

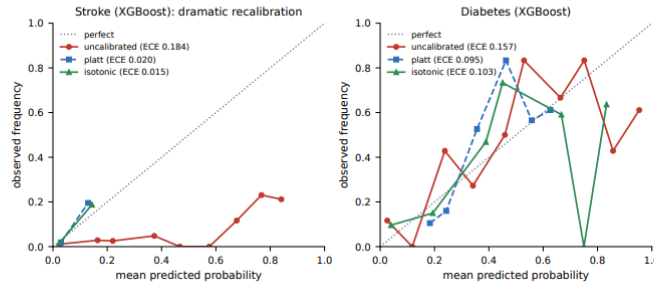


Figure 3: Measured reliability curves for the designated gradient-boosted model. On the rare-positive stroke cohort (left) the raw scores depart the diagonal and isotonic and Platt recalibration map them back onto calibrated probabilities, cutting the expected calibration error from 0.184 to 0.015 without changing the ranking; the diabetes panel (right) shows the milder correction Platt applies there.

The design rationale is that for screening the absolute probability must be trustworthy, not merely the ranking, so calibration is reported beside discrimination. Trusting the raw score is unsafe precisely when discrimination is high: a sharp model with mis-stated probabilities refers the wrong fraction of patients at any fixed threshold. Fitting g on a held-out calibration split avoids an optimistic map that looks calibrated in sample, and reading the reliability metrics on the still-separate test split avoids the symmetric optimism of scoring g on the data it was fit to. We compare isotonic and Platt because they trade flexibility against data efficiency differently: reporting both, with Brier, ECE, and the reliability curve, lets the trade-off be read rather than assumed. Because recalibration is a monotone transform, it leaves the ranking and the AUC unchanged, so a drop in the calibration-specific ECE at fixed AUC isolates a pure calibration gain, which is why we report discrimination and calibration as separate axes and lean on this AUC-invariance rather than on the Brier score alone.

3.7 TreeSHAP Explanation, Protocol, and Pseudocode

The fourth stage explains the gradient-boosted model with additive feature attributions, so each prediction comes with a defensible account of its drivers. TreeSHAP explains the uncalibrated boosted score $s = g^{-1}(\hat{p})$ rather than the calibrated probability, since g is applied after the trees and TreeSHAP attributes the tree-ensemble output. This costs nothing in consistency: because g is monotone it preserves the score order and so leaves the SHAP driver ranking unchanged, keeping the explanation of s and the calibrated $\hat{p}$ consistent. SHAP assigns to each feature j a value ϕ_j such that the attributions and a base value ϕ_0 reconstruct the model output for a patient,

$$g^{-1}(\hat{p}) = \phi_0 + \sum_{j \in S_k} \phi_j, \quad (3)$$

Figure 9: Paper Demo (Part 3): Screenshot for the demo paper experiment and analysis.

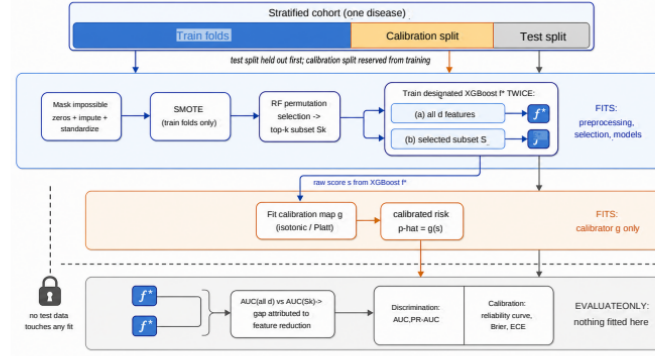


Figure 4: The feature-selection comparison protocol: the gradient-boosted model is trained twice per cohort, once on all features and once on the random-forest-permutation-selected top-k subset, and the discrimination gap is attributed to the reduction in feature count, all within the leakage-safe folds.

random forest and the SVM, so Accuracy and F1 are both deceptive here. The informative stroke metric is therefore PR-AUC, which reached 0.157 for the boosted ensemble, 0.146 for the random forest, 0.123 for the perceptron, and 0.115 for the SVM, roughly three times the 0.049 base rate and a real, if modest, lift over chance. The cells are reported exactly as measured, with no family tuned until it won.

Table 2: Primary discrimination comparison across cohorts: the four model families on Accuracy, Precision, Recall, F1, AUC, and PR-AUC under repeated stratified cross-validation. Higher is better for all metrics; Precision/Recall/F1 use the 0.5 operating point on the calibrated probability. Cells report the measured mean $\pm$ SD; the best AUC per cohort is bold.

Cohort	Model	Accuracy	Precision	Recall	F1	AUC	PR-AUC
Diabetes	RF	0.739 $\pm$ 0.032	0.652 $\pm$ 0.066	0.580 $\pm$ 0.148	0.597 $\pm$ 0.093	0.811 $\pm$ 0.029	0.669 $\pm$ 0.057
Diabetes	XGBoost	0.726 $\pm$ 0.035	0.677 $\pm$ 0.085	0.469 $\pm$ 0.175	0.525 $\pm$ 0.128	0.793 $\pm$ 0.031	0.666 $\pm$ 0.045
Diabetes	SVM	0.749 $\pm$ 0.028	0.687 $\pm$ 0.091	0.584 $\pm$ 0.134	0.612 $\pm$ 0.057	0.826$\pm$0.031	0.690 $\pm$ 0.054
Diabetes	MLP	0.723 $\pm$ 0.035	0.651 $\pm$ 0.099	0.509 $\pm$ 0.133	0.554 $\pm$ 0.073	0.794 $\pm$ 0.046	0.651 $\pm$ 0.079
Heart disease	RF	0.776 $\pm$ 0.078	0.809 $\pm$ 0.121	0.711 $\pm$ 0.119	0.744 $\pm$ 0.085	0.869 $\pm$ 0.054	0.874 $\pm$ 0.056
Heart disease	XGBoost	0.770 $\pm$ 0.043	0.779 $\pm$ 0.083	0.730 $\pm$ 0.129	0.741 $\pm$ 0.063	0.845 $\pm$ 0.061	0.839 $\pm$ 0.075
Heart disease	SVM	0.800 $\pm$ 0.047	0.818 $\pm$ 0.084	0.746 $\pm$ 0.072	0.775 $\pm$ 0.047	0.873$\pm$0.053	0.872 $\pm$ 0.051
Heart disease	MLP	0.737 $\pm$ 0.054	0.751 $\pm$ 0.084	0.679 $\pm$ 0.153	0.697 $\pm$ 0.080	0.848 $\pm$ 0.059	0.844 $\pm$ 0.067
Stroke	RF	0.951 $\pm$ 0.002	0.022 $\pm$ 0.083	0.002 $\pm$ 0.008	0.004 $\pm$ 0.014	0.795$\pm$0.030	0.146 $\pm$ 0.024
Stroke	XGBoost	0.950 $\pm$ 0.003	0.172 $\pm$ 0.339	0.013 $\pm$ 0.025	0.021 $\pm$ 0.039	0.793 $\pm$ 0.033	0.157 $\pm$ 0.024
Stroke	SVM	0.950 $\pm$ 0.002	0.017 $\pm$ 0.062	0.002 $\pm$ 0.008	0.004 $\pm$ 0.014	0.747 $\pm$ 0.041	0.115 $\pm$ 0.023
Stroke	MLP	0.950 $\pm$ 0.002	0.139 $\pm$ 0.275	0.011 $\pm$ 0.019	0.019 $\pm$ 0.032	0.731 $\pm$ 0.039	0.123 $\pm$ 0.022

The calibration comparison is reported in Table 3, which lists the Brier score and the expected calibration error of the designated gradient-boosted model per cohort under the raw scores and after Platt and isotonic recalibration, with the map fitted on the calibration split and the metrics read on the test split. Recalibration helped, and most dramatically exactly where it matters: on the rare-positive stroke cohort the raw scores were badly miscalibrated, a Brier of 0.132 and an ECE of 0.184, and isotonic recalibration cut these to a Brier of 0.042 and an ECE of 0.015, an order-of-magnitude improvement in the calibration error, with Platt close behind at a Brier of 0.043 and an ECE of 0.020. On diabetes, Platt was the better map, lowering the Brier from 0.196 to 0.182 and the ECE from 0.157 to 0.095, while isotonic also helped but less, to a Brier of 0.189 and an ECE of 0.103. The heart-disease cohort exposed an honest small-cohort instability: with only forty-nine calibration-split points the parametric Platt map actually *hurt* the ECE, raising it from a raw 0.089 to 0.197 and the

Figure 10: Paper Demo (Part 4): Screenshot for the demo paper experiment and analysis.